

Podstawy uczenia maszynowego

klasteryzacja

ćwiczenia nr 7

Akademia Górniczo-Hutnicza
im. Stanisława Staszica w Krakowie
AGH University of Krakow

semestr letni 2025/2026

Paweł Szczerbiak

Czym się zajmiemy na ćwiczeniach?

Ćwiczenia 1: metody liniowe

Ćwiczenia 2: metody oparte na sąsiedztwie

Ćwiczenia 3: metody jądrowe

Ćwiczenia 4: metody probabilistyczne

Ćwiczenia 5: klasyfikacja niezbalansowana

Ćwiczenia 6: selekcja cech i redukcja wymiarowości

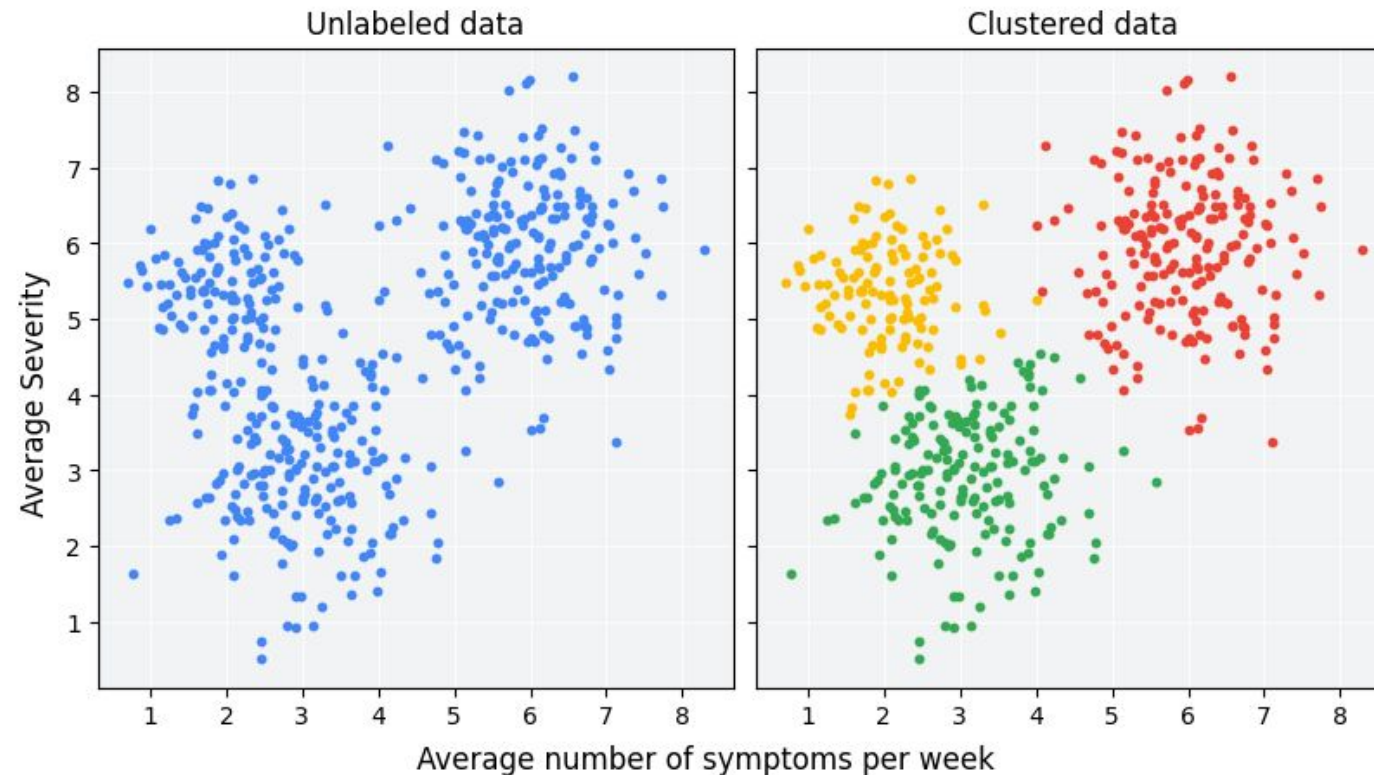
Ćwiczenia 7: klasteryzacja

Zagadnienia

1. Klasteryzacja
 - a. zastosowanie
 - b. metody
2. k-means
3. k-means++
4. k-medoids
5. DBSCAN
6. HDBSCAN
7. Klastrowanie hierarchiczne
8. Co dalej? Moje rekomendacje

Klasteryzacja

- **Grupowanie** punktów względem ich **podobnej** charakterystyki
- Klasteryzacja, podobnie jak redukcja wymiarowości, jest ważnym przykładem uczenia nienadzorowanego (*unsupervised learning*) – o tym dalej
- Analizujemy cały zbiór, bez podziału na treningowy / walidacyjny / testowy



Klasteryzacja – zastosowanie

- Analiza danych (*exploratory data analysis*, EDA)
 - lepsze zrozumienie danych jako finalny cel
 - odkrycie potencjalnych podklas, które możemy użyć do treningu
 - wykrywanie punktów odstających lub całkowicie nowych danych (*outlier / novelty / anomaly detection*)
- Segmentacja klientów (*customer segmentation*)
 - dzielenie za pomocą ich cech (wymagana selekcja cech)
 - tworzenie spersonalizowanych kampanii reklamowych
- Systemy rekomendacyjne
 - identyfikacja grup ludzi o podobnym guście
 - rekomendacja tego, co podobało się innym
- Odkrywanie tematów tekstów
- Biologia / genetyka
 - tworzenie drzew filogenetycznych
 - znajdowanie genów i białek o podobnej funkcji (zastosowanie medyczne)

Klasteryzacja – metody

- **Oparta na podziałach** (*partition-based*):
 - dzielimy dane na k niepokrywających się klastrów
 - przykłady: k-means, k-medoids
 - **zalety**: szybka i wydajna, łatwa do zaimplementowania
 - **wady**: wymaga podania k , wrażliwa na wartości początkowe i punkty odstające, zakłada sferyczne klastry o podobnej wielkości
- **Oparta na gęstości** (*density-based*):
 - tworzy klastry jako obszary o wysokiej gęstości oddzielone obszarami o niskiej gęstości
 - przykłady: DBSCAN, HDBSCAN, OPTICS
 - **zalety**: działa z klastrami o dowolnym kształcie, odporna na szum i punkty odstające
 - **wady**: problemy przy zmiennej gęstości danych, wymaga ostrożnego doboru parametrów

Klasteryzacja – metody

- **Hierarchiczna** (*hierarchical*):
 - agregacyjna (*agglomerative*): zaczyna od pojedynczych punktów i łączy je w klastry
 - dzieląca (*divisive*): zaczyna od jednego klastra i dzieli go na mniejsze
 - **zalety**: dendrogram (wynik klastrowania) pozwala wizualizować strukturę danych
 - **wady**: kosztowna obliczeniowo dla dużych zbiorów danych, wymaga zdefiniowania, jak zrobić podział
- **Oparta na modelach** (*model-based*):
 - zakłada pewną strukturę danych i na tej podstawie dokonuje podziału
 - przykłady: GMM, LDA
 - **zalety**: elastyczna, interpretowalna
 - **wady**: zakładany model może być błędny

Klasteryzacja – metody

- **Oparta na grafach** (*graph-based*):
 - tworzy graf podobieństwa między punktami danych
 - przykład: spektralna (*spectral*)
 - **zalety**: radzi sobie z nieregularnymi, niesferycznymi klastrami, dobrze działa przy słabo rozdzielonych klastrach
 - **wady**: wymaga dużej mocy obliczeniowej, wymaga z góry podanej liczby klastrów
- **Oparta na siatkach** (*grid-based*):
 - dzieli przestrzeń danych na komórki siatki i przeprowadza klasteryzację na ich podstawie
 - przykłady: STING, CLIQUE
 - **zalety**: szybka, szczególnie dla danych przestrzennych
 - **wady**: zależna od rozdzielczości siatki, mniej popularna

k-means

Funkcja kosztu (minimalizacja wariancji w obrębie klastrów):

$$\arg \min_{\mathbf{S}} \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2 = \arg \min_{\mathbf{S}} \sum_{i=1}^k |S_i| \text{Var } S_i$$

centroidy

$$\boldsymbol{\mu}_i = \frac{1}{|S_i|} \sum_{\mathbf{x} \in S_i} \mathbf{x}$$

tożsame z maksymalizacją wariancji między klastrami (wynika z prawa całkowitej wariancji, która jest stała)

Algorytm (nieoptymalny, naiwny):

1. Wybierz początkowe k centroidów (reprezentantów klastrów), np. jako losowe punkty ze zbioru
2. Powtarzaj aż do osiągnięcia zbieżności:
 - a. przyporządkuj każdy punkt do najbliższego centroidu
 - b. zaktualizuj koordynaty centroidów jako średnią z przypisanych do niego punktów

k-means++

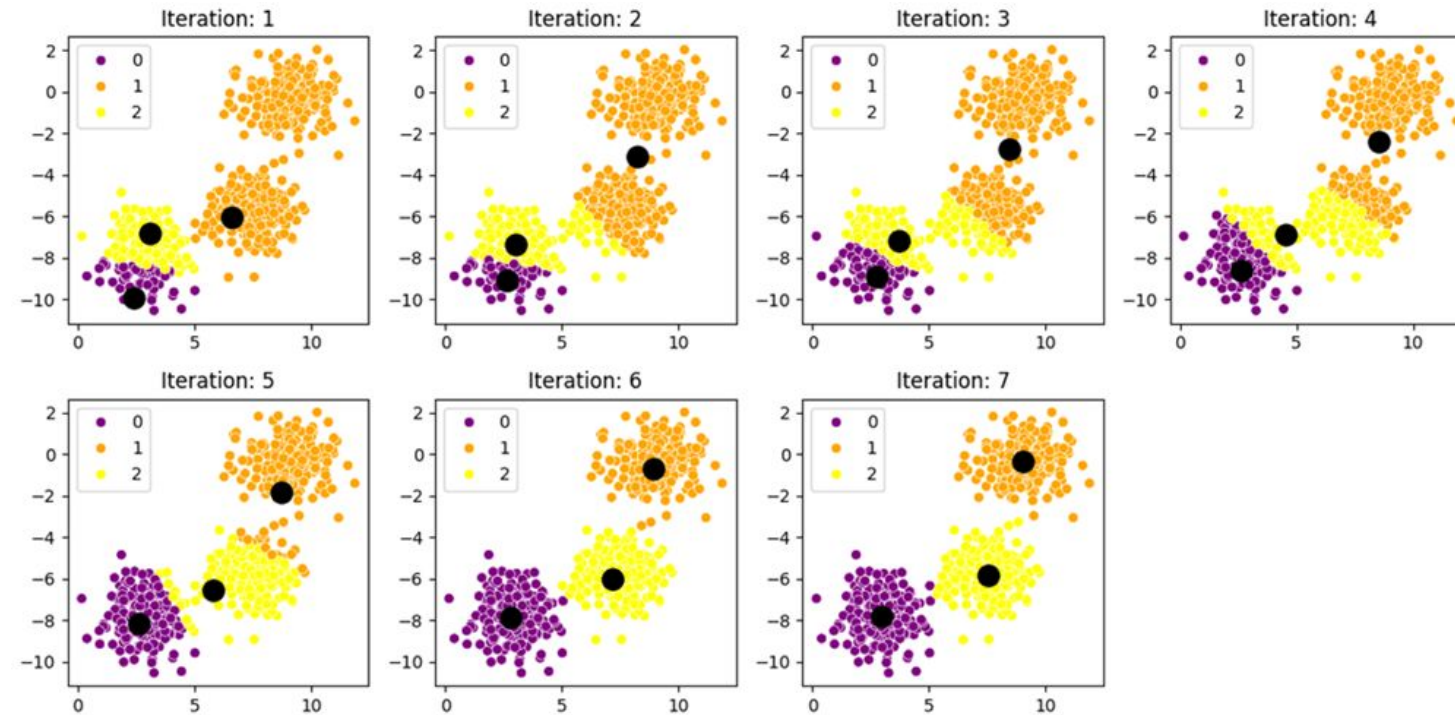
- Inaczej wybiera początkowe centroidy
- Domyślna wersja k-means w `scikit-learn`

Algorytm:

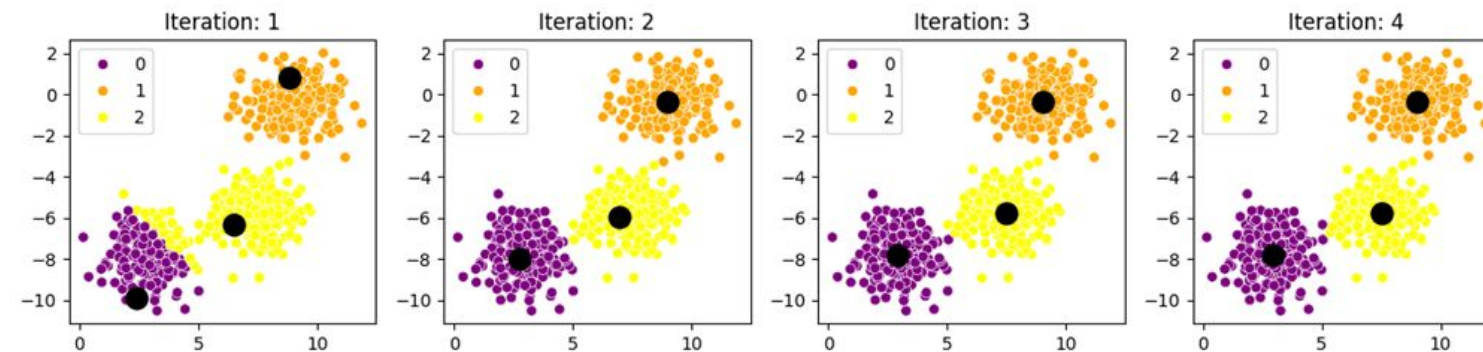
1. Wybierz losowo pierwszy centroid
2. Wybierz kolejne centroidy według schematu (aż do uzyskania k centroidów):
 - a. oblicz odległość $d(x)$ każdego punktu danych od najbliższego centroidu
 - b. przypisz prawdopodobieństwo do każdego punktu na podstawie $d^2(x)$, przy czym dalsze punkty mają większą szansę na wybór
 - c. Wylosuj centroid na podstawie tego rozkładu prawdopodobieństwa
3. Kontynuuj według algorytmu k-means

k-means vs k-means++

k-means



k-means++



k-means – podsumowanie

Zalety:

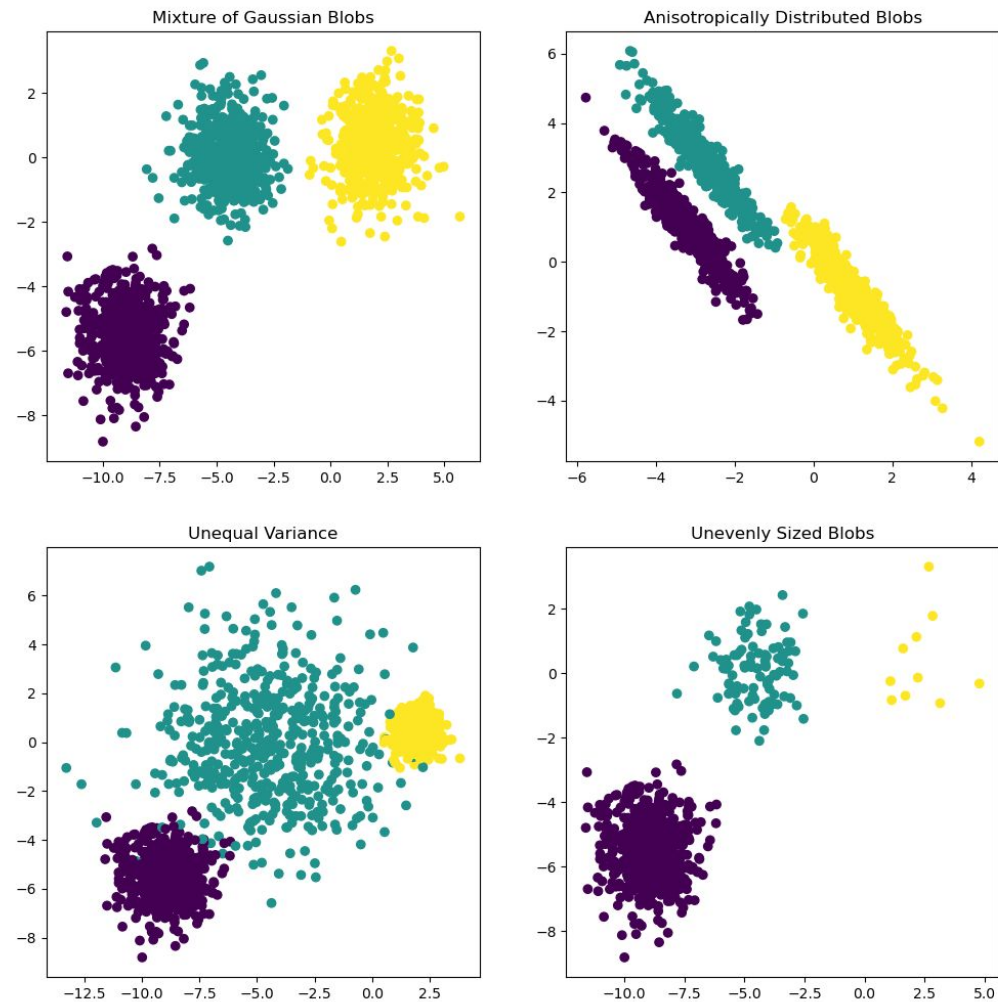
- prostota
- niski koszt obliczeniowy
- potrafi robić predykcję (na podstawie bliskości do centroidów)

Wady:

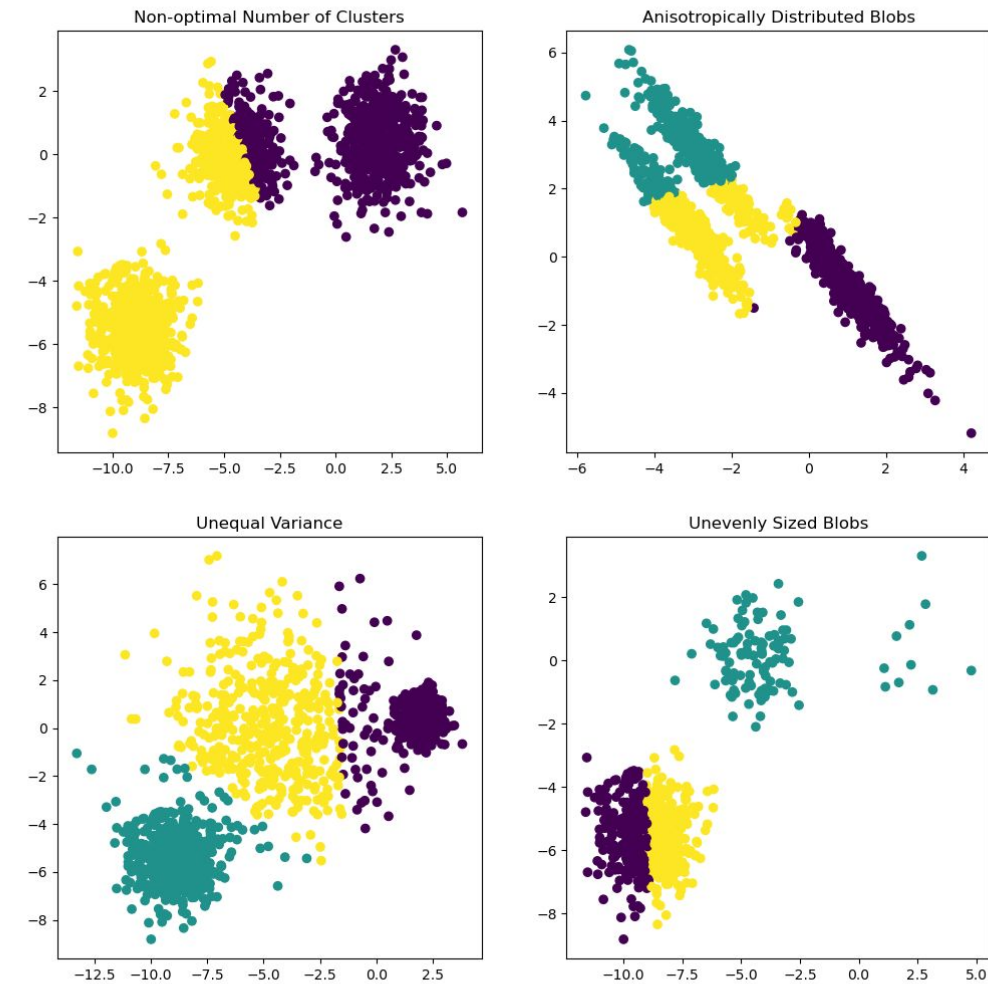
- zakłada normalny (sferyczny) rozkład punktów w klastrach
- **czuły na punkty odstające i kłutwę wymiarowości (wymagana redukcja wymiarowości)**
- prowadzi do klastrów podobnych rozmiarów
- tylko metryka euklidesowa
- centroidy to nieistniejące punkty
- wymaga normalizacji danych (liczymy odległości)

k-means – podsumowanie

wynik oczekiwany

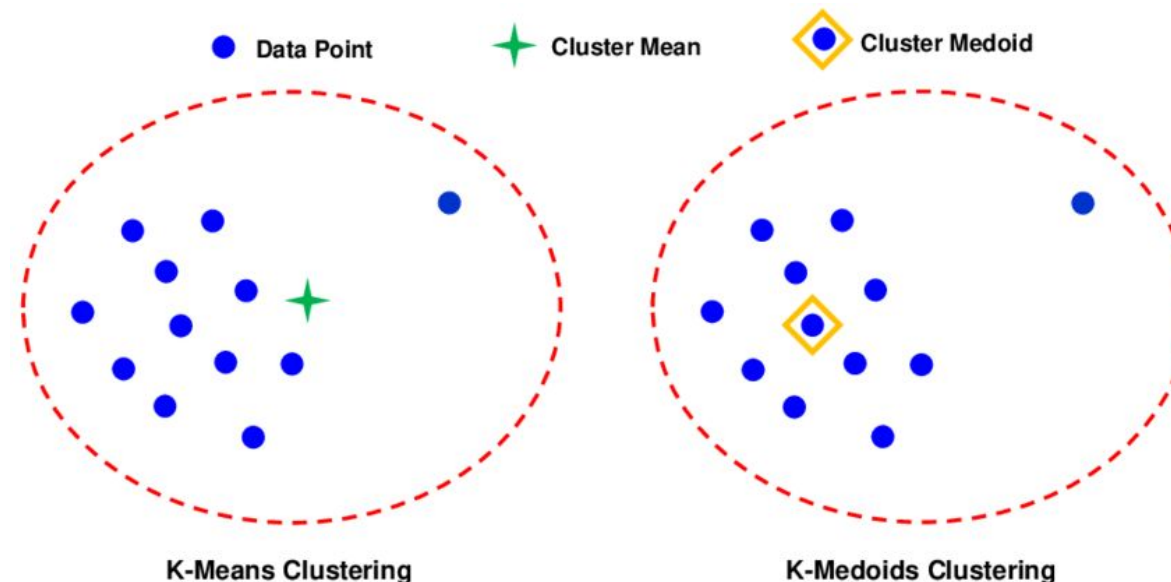


wynik klastrowania



k-medoids

- Jako centroid bierzemy **rzeczywisty** punkt w danych (tzw. medoid)
- Pozwala na **dowolną metrykę** (nie ma funkcji kosztu jak k-means)
- Mniej czuły na punkty odstające niż k-means
- Większy koszt obliczeniowy niż w k-means
- Pozostałe wady (sferyczne klastry, dobór liczby klastrów) i zalety (prostota predykcja) jak w k-means



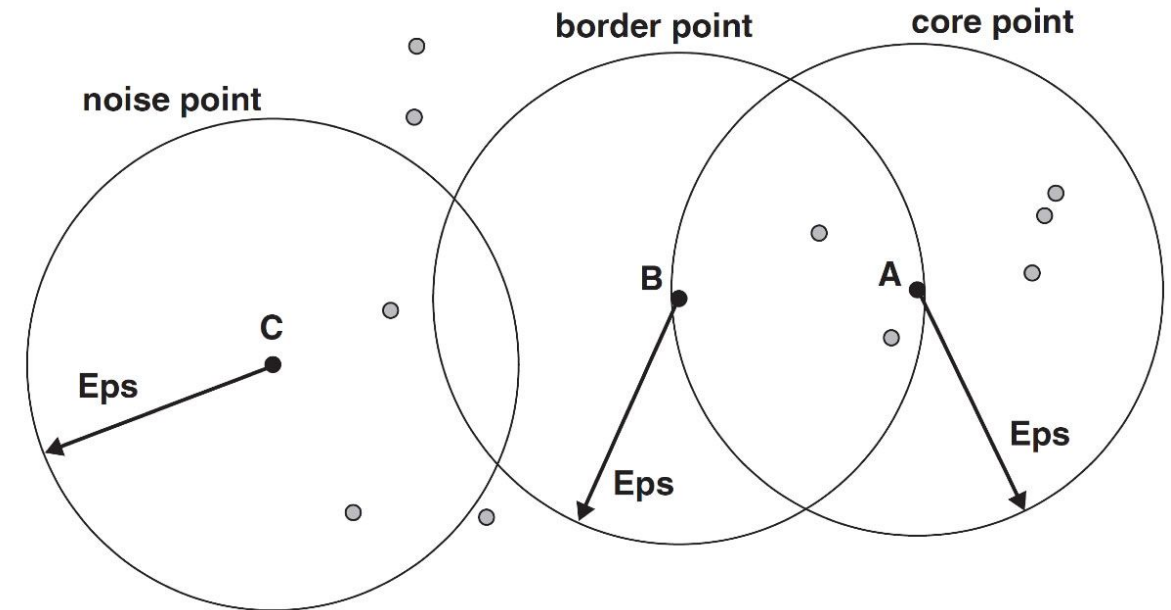
DBSCAN

Density-Based Spatial Clustering for Applications with Noise

- Klastry to gęste obszary rozdzielone rzadkimi obszarami
- Nie każdy punkt musi być w klastrze (zawsze jest jakiś szum)

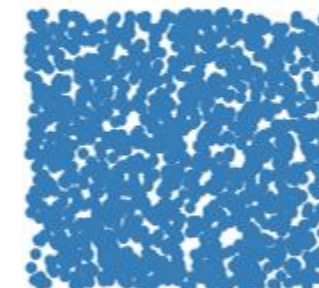
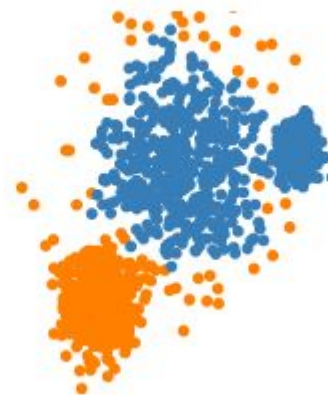
Algorytm:

1. Dla każdego punktu znajdź wszystkie punkty w promieniu *Epsilon*
2. Oznacz punkty jako *Core*, *Border* i *Noise* w zależności od hiperparametru *MinPoints*
3. Stwórz klastry, łącząc punkty *Core*
4. Dołącz punkty *Border* do sąsiedniego klastra

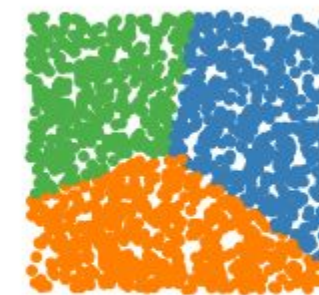
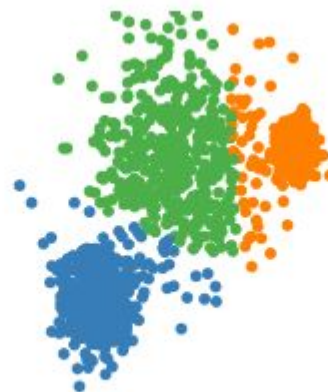


DBSCAN vs k-means

DBSCAN



k-means



DBSCAN – podsumowanie

Zalety:

- prostota
- niski koszt obliczeniowy
- sam dobiera liczbę klastrów
- klastry mogą być dowolnych kształtów

Wady:

- ciężko dobrać hiperparametry
- kiepski przy zróżnicowanej gęstości (globalne hiperparametry)
- podatny na klątwę wymiarowości (bazuje na najbliższych sąsiadach)
- nie robi predykcji

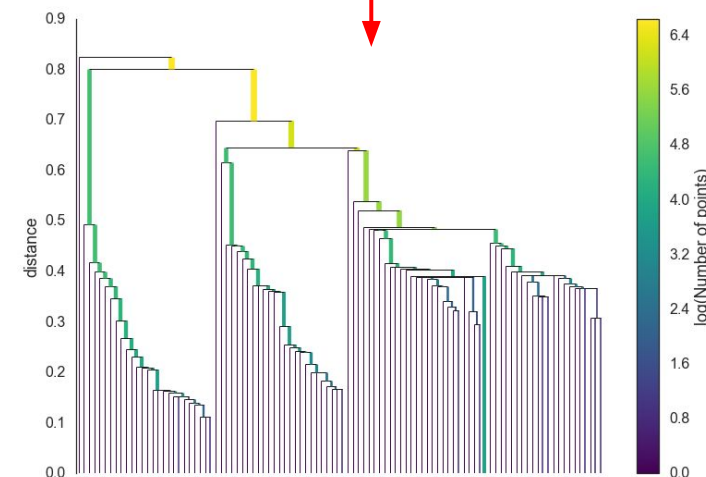
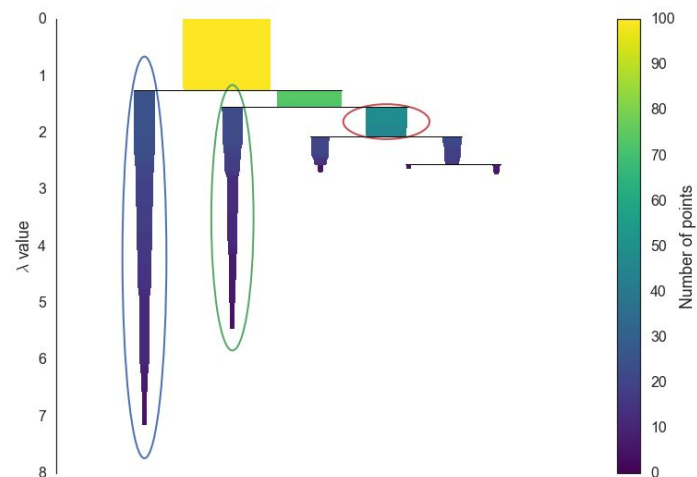
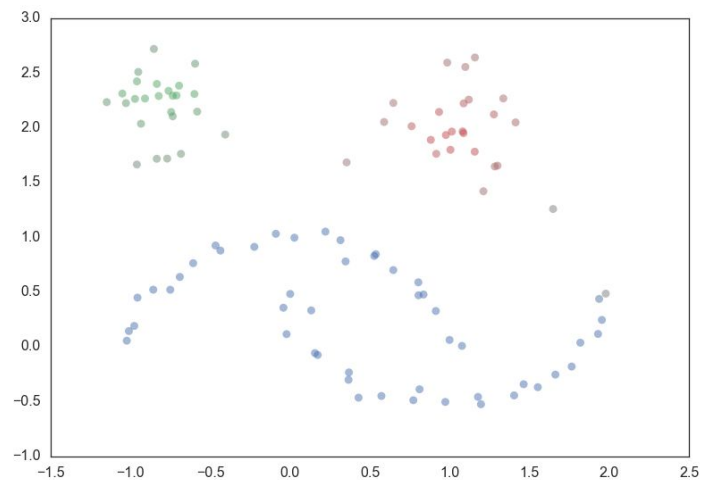
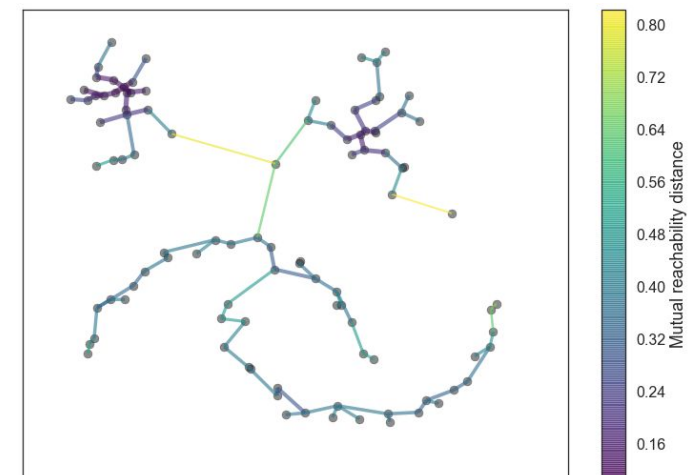
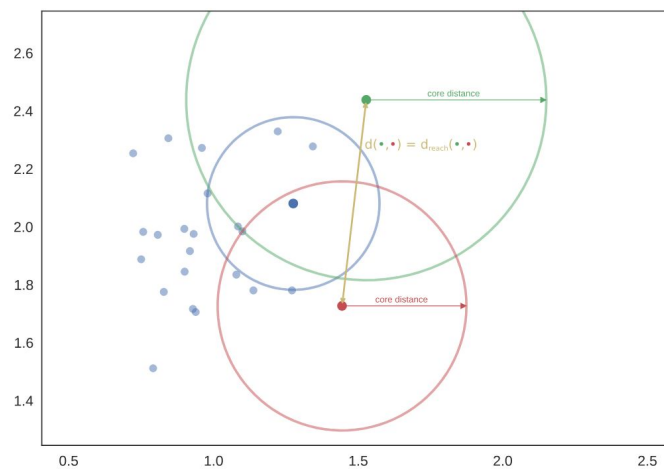
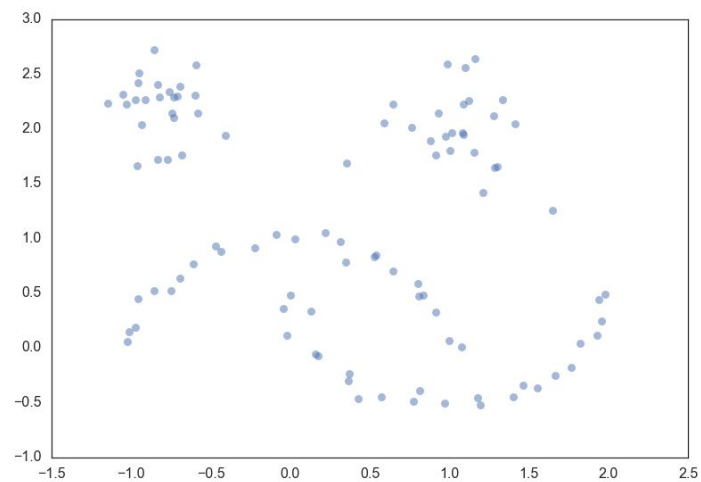
Hierarchical DBSCAN

- Buduje hierarchię (dendrogram) przedstawiającą klastry na różnych poziomach gęstości
- W przeciwieństwie do DBSCAN **może wykrywać klastry o różnej gęstości**
- Jest też od niego szybszy i bardziej odporny na szum, ma też proste hiperparametry

Algorytm:

1. Przekształć odległości między punktami na miarę związaną z gęstością (*mutual reachability distance*), która wygładza różnice w gęstości
2. Zbuduj minimalne drzewo rozpinające (MST) dla wszystkich punktów z użyciem tej miary
3. Stwórz hierarchię klastrów (dendrogram), stopniowo usuwając krawędzie z MST
4. Wykonaj płaską klasteryzację, wybierając najbardziej stabilne klastry z hierarchii
5. Oznacz punkty, które nie pasują do żadnego stabilnego klastra, jako szum

HDBSCAN



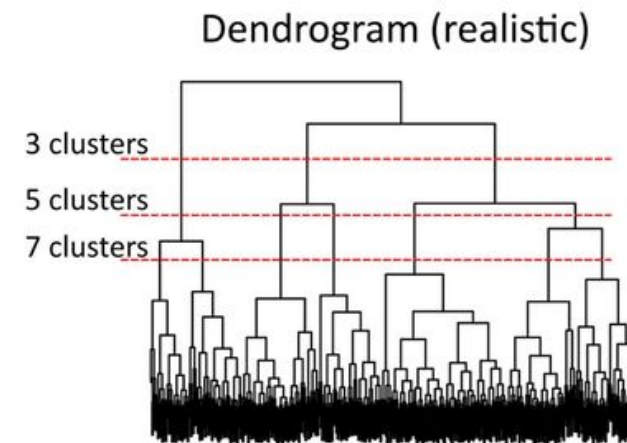
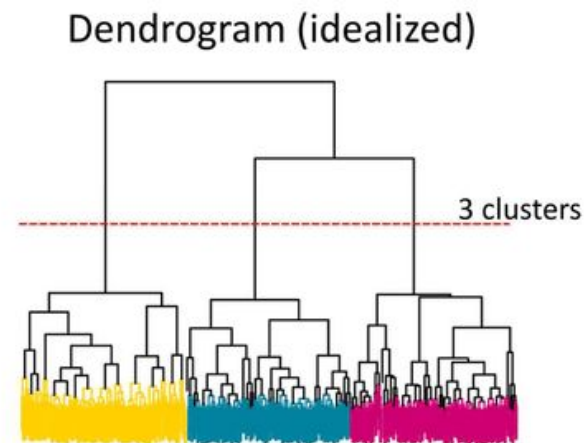
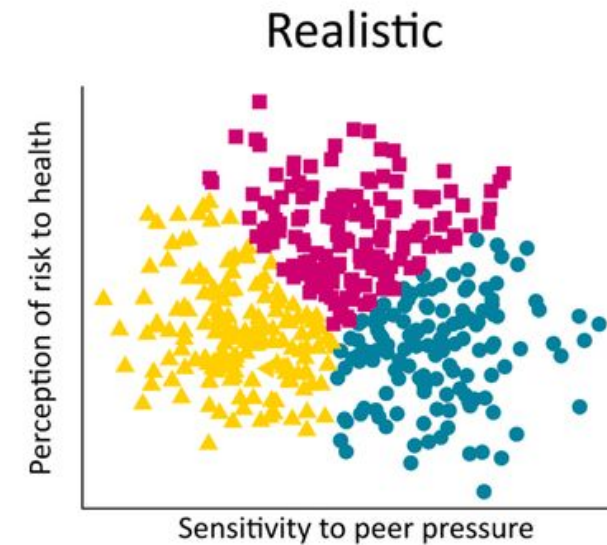
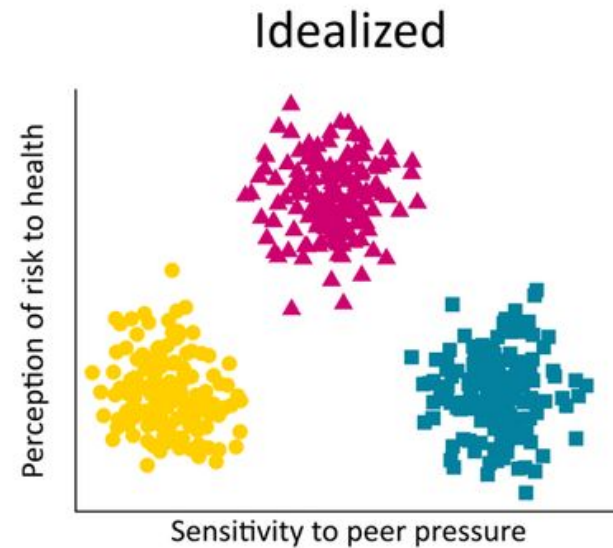
Klastrowanie hierarchiczne

- **Idea:** duże klastry mogą się składać z mniejszych
- Metody hierarchiczne tworzą hierarchię klastrow
- Trudnością tego podejścia jest wyodrębnienie płaskich klastrow (jest to zawsze arbitralne)

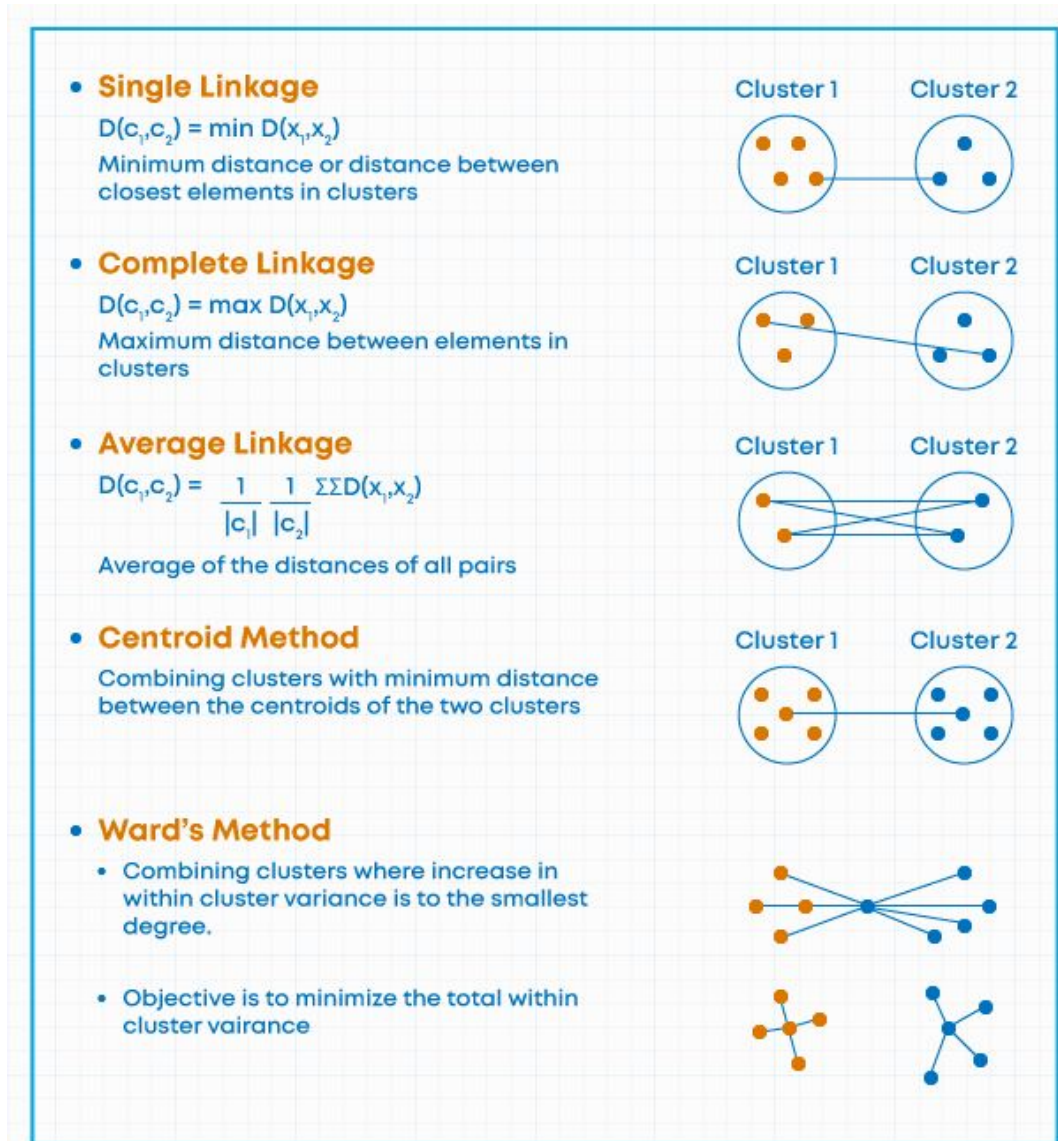
Algorytm (podejście dół-góra tj. *bottom-up*):

1. Potraktuj każdy punkt jako 1-elementowy klaster
2. Powtarzaj poniższe kroki aż do uzyskania jednego klastra:
 - a. Oblicz odległości między klastrami (każdy z każdym)
 - b. Znajdź dwa najbliższe klastry
 - c. Połącz je
 - d. Zapamiętaj odległość między klastrami
3. Stwórz dendrogram (drzewo) za pomocą odległości

Klastrowanie hierarchiczne – dendrogram



Klastrowanie hierarchiczne – łączenie klastrów



szybkie, ale podatne na szum

szybkie, ale podatne na szum

odporne na szum, ale duży koszt obliczeniowy

produkuje zbalansowane klastry, ale podatne na punkty odstające

bardzo odporne na szum, ale duży koszt obliczeniowy

Klastrowanie hierarchiczne – łączenie klastrów

- **Single Linkage**
 $D(c_1, c_2) = \min D(x_1, x_2)$
Minimum distance or distance between closest elements in clusters
- **Complete Linkage**
 $D(c_1, c_2) = \max D(x_1, x_2)$
Maximum distance between elements in clusters
- **Average Linkage**
 $D(c_1, c_2) = \frac{1}{|c_1|} \frac{1}{|c_2|} \sum \sum D(x_1, x_2)$
Average of the distances of all pairs
- **Centroid Method**
Combining clusters with minimum distance between the centroids of the two clusters
- **Ward's Method**
 - Combining clusters where increase in within cluster variance is to the smallest degree.
 - Objective is to minimize the total within cluster variance

Cluster 1

Cluster 2



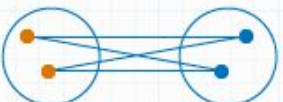
Cluster 1

Cluster 2



Cluster 1

Cluster 2

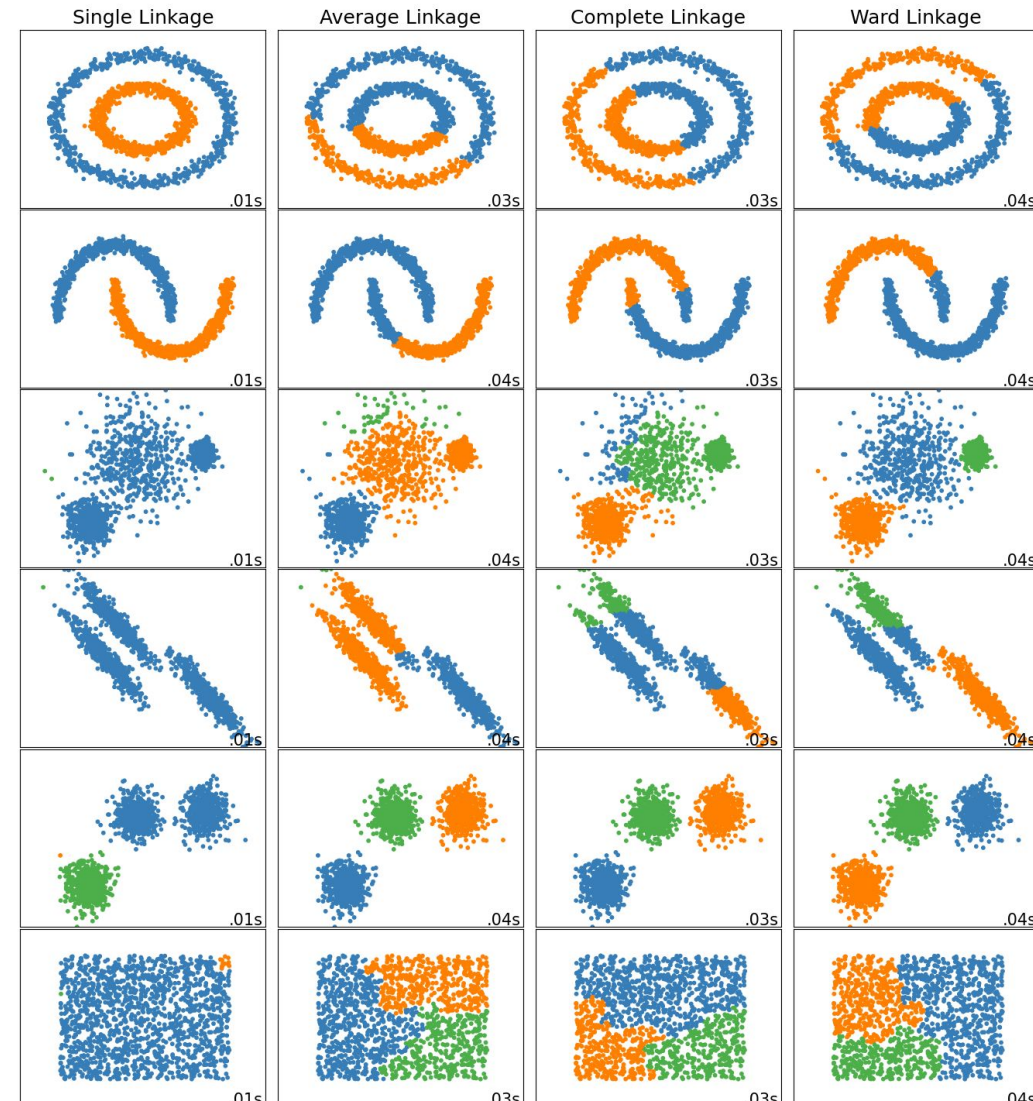


Cluster 1

Cluster 2



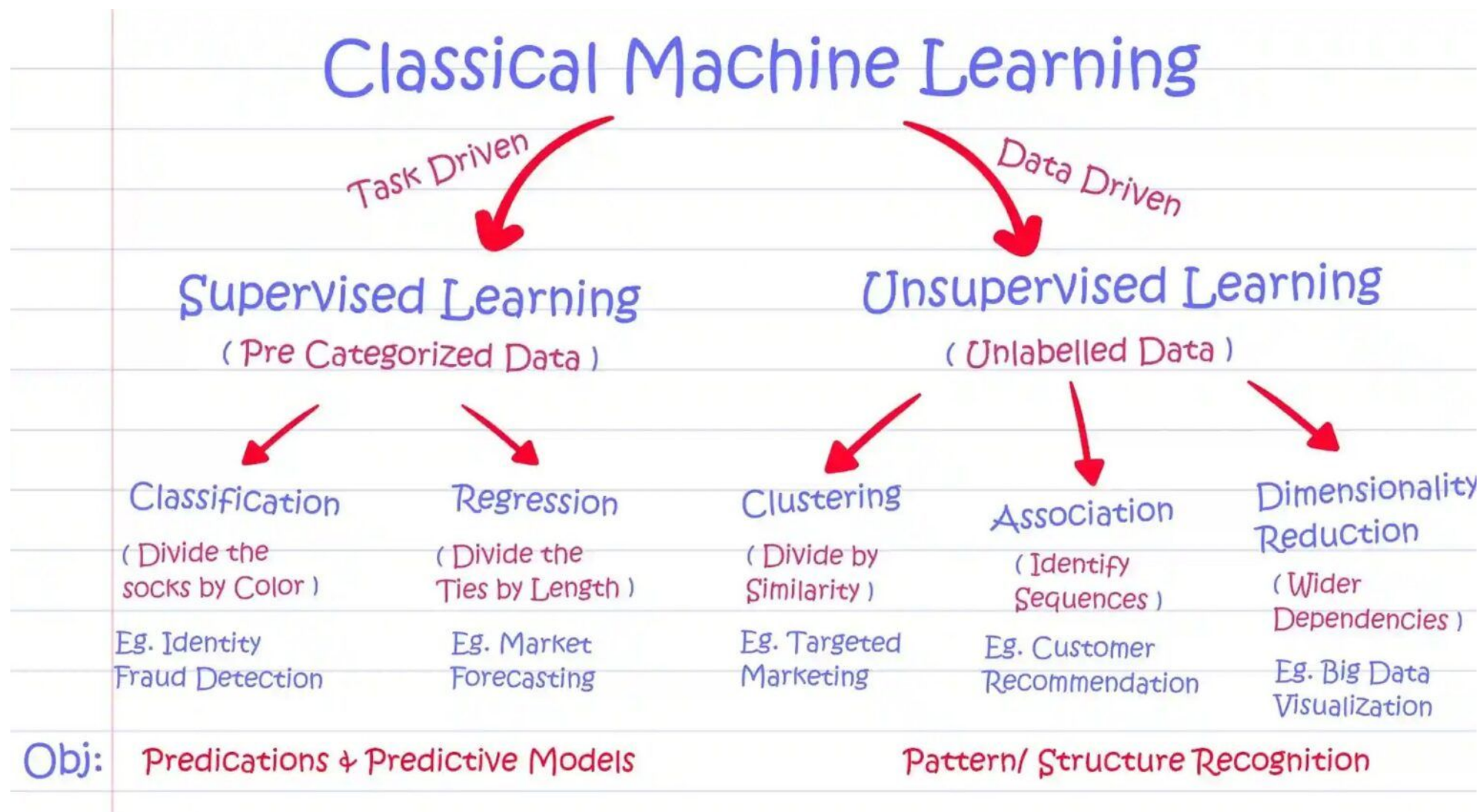


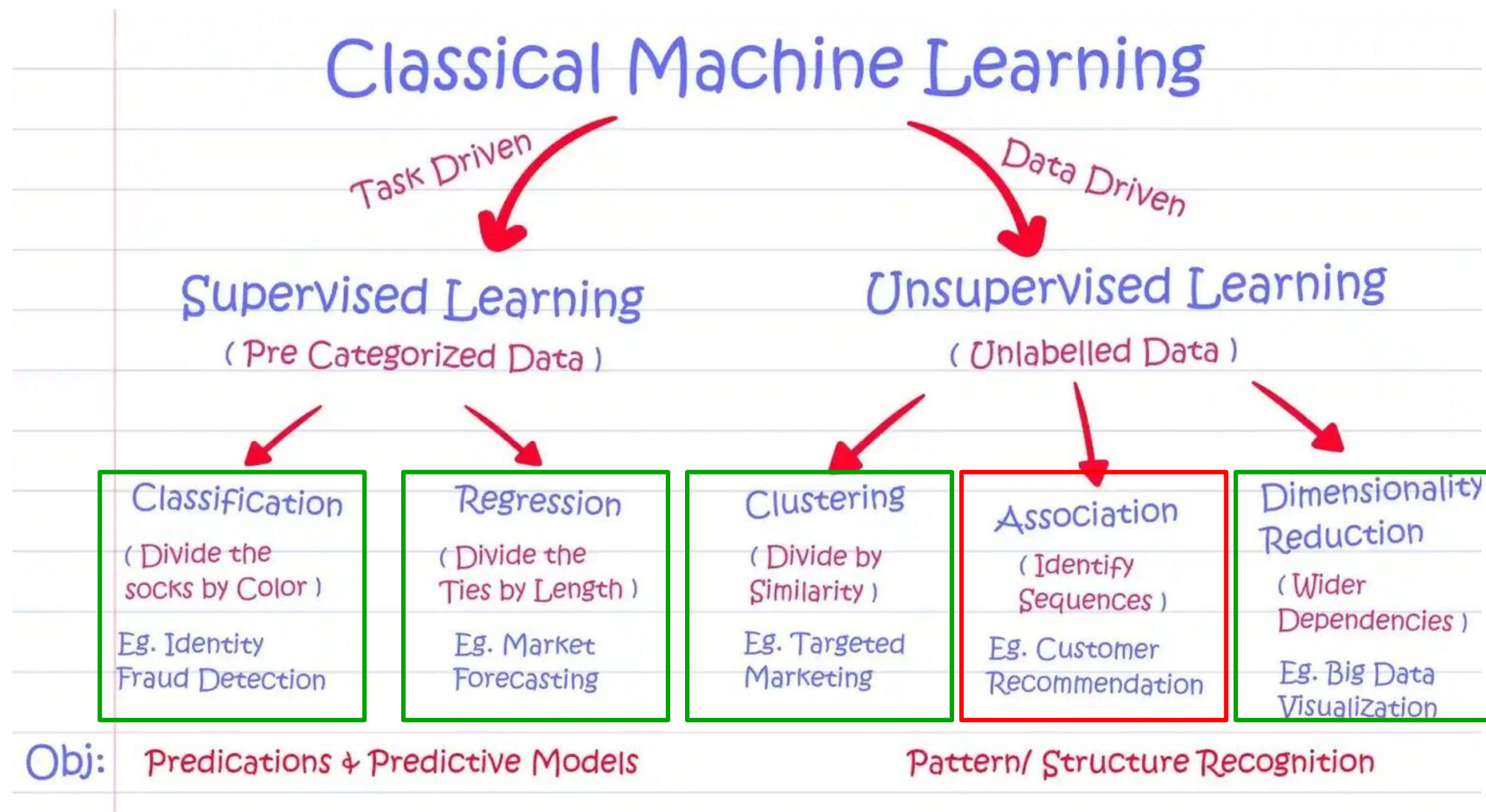
Do zapamiętania

1. Klasteryzacja i jej zastosowanie
2. Metody:
 - a. Oparte na podziałach
 - b. Oparte na gęstości
 - c. Hierarchiczne
 - d. Oparte na modelach
 - e. Oparte na grafach
 - f. Oparte na siatce
3. k-means i k-means++
4. k-medoids
5. DBSCAN i HDBSCAN
6. Klastrowanie hierarchiczne

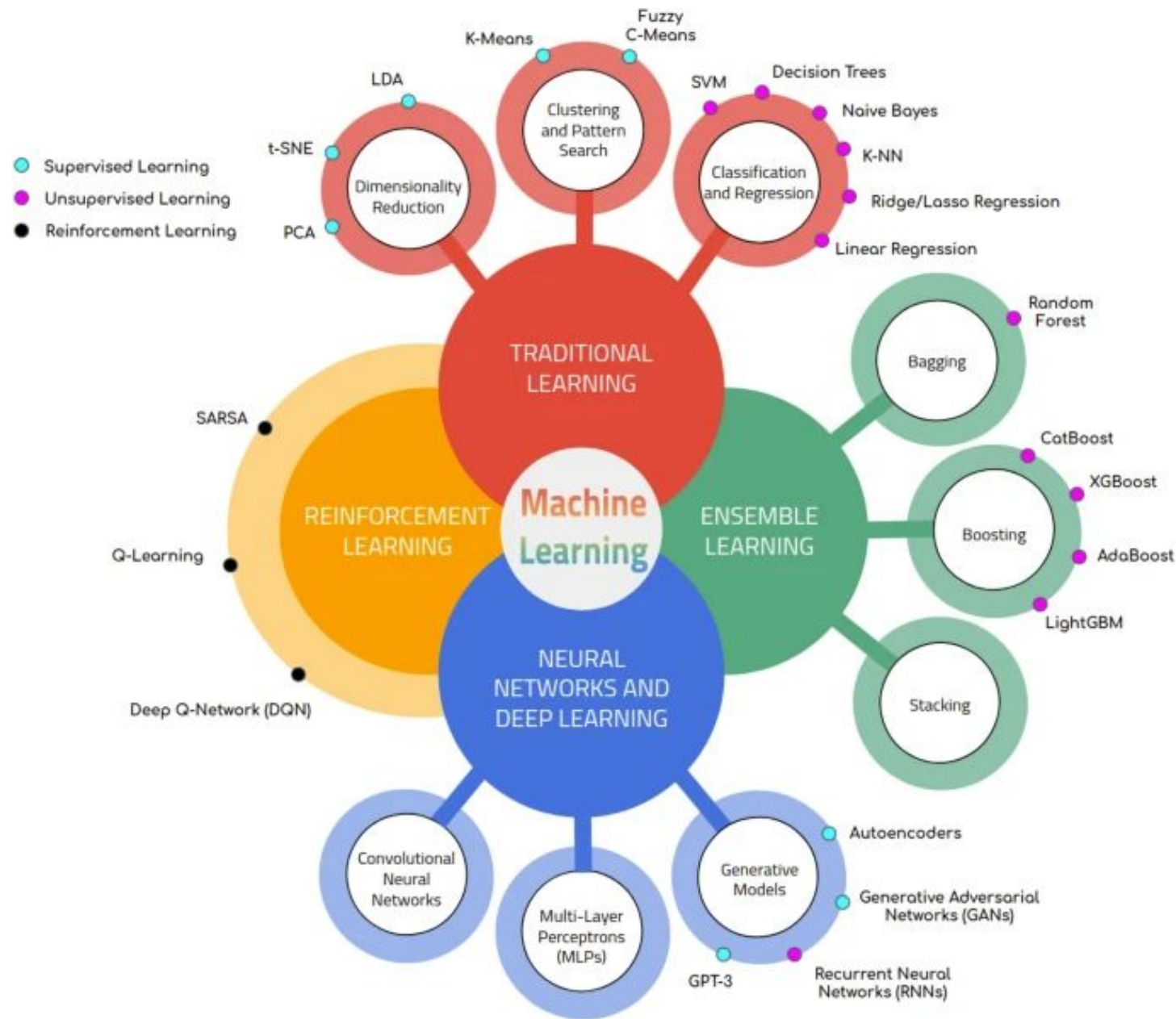
Co dalej?



Co dalej?

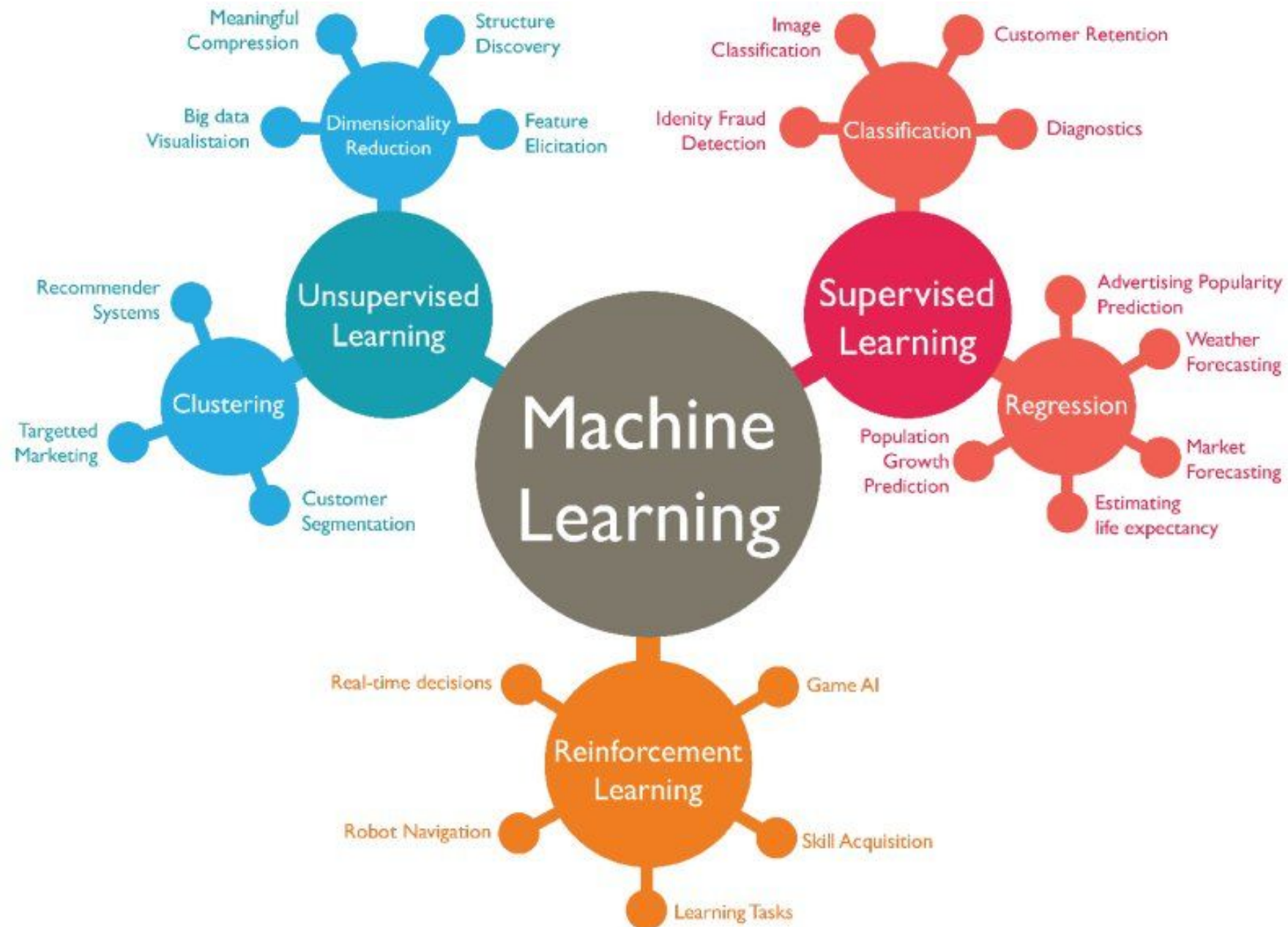


Co dalej?



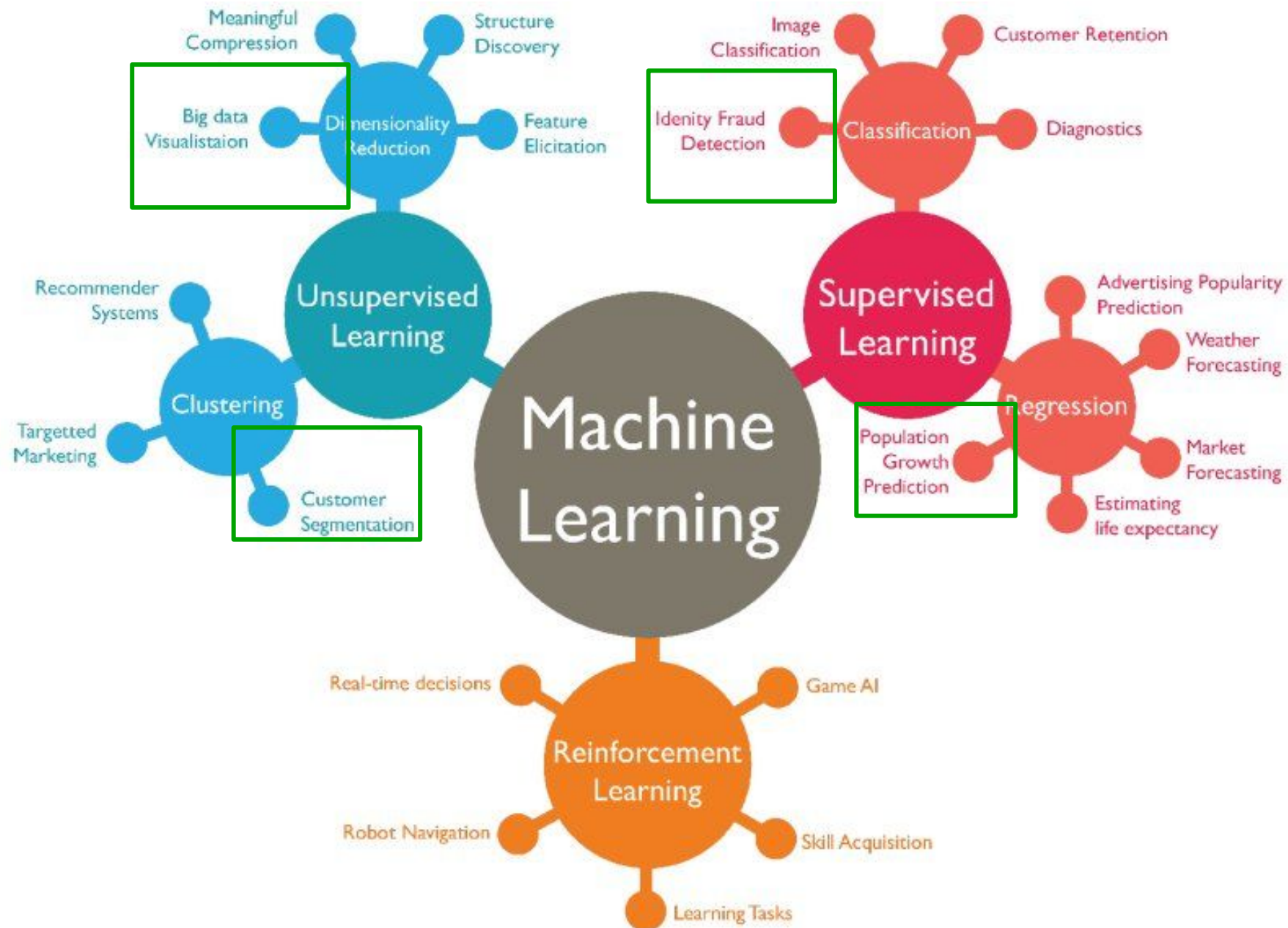
Artificial intelligence: machine learning for chemical sciences

Co dalej?



Co dalej?

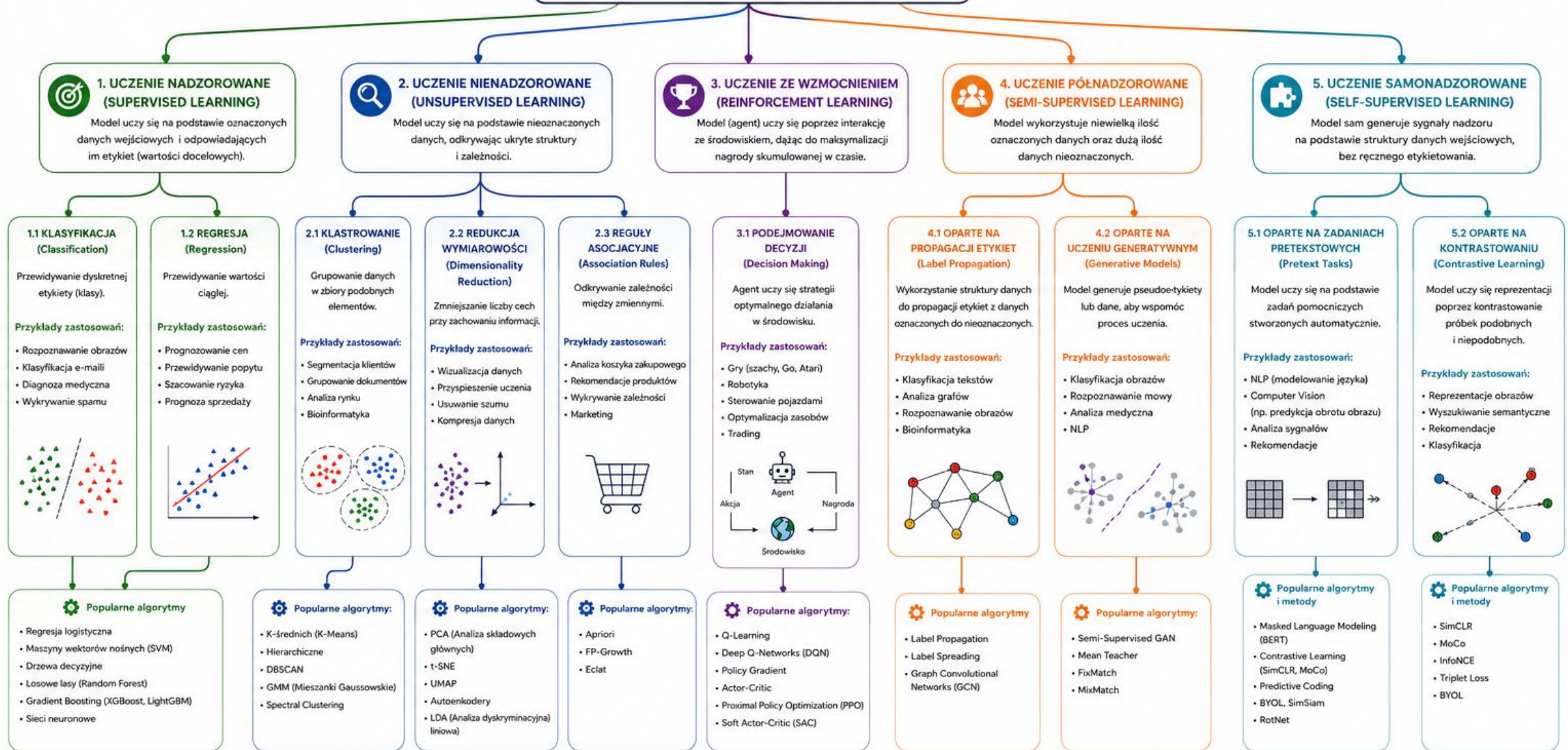
to tylko przykłady





PARADYMATY W UCZENIU MASZYNOWYM

Uczenie maszynowe to dziedzina sztucznej inteligencji, która umożliwia systemom uczenie się na podstawie danych i poprawę wydajności bez jawnego programowania.



KLUCZOWA IDEA

Każdy paradygmat wykorzystuje inne informacje dostępne w danych i inne mechanizmy uczenia, aby rozwiązywać różne problemy. W praktyce granice między paradygmatami mogą się zacierać, a wiele metod łączy elementy kilku podejść.

LEGENDA

- Zielony – wykorzystuje etykiety (pełny nadzór)
- Niebieski – odkrywa struktury w danych (brak etykiet)
- Fioletowy – uczy się przez interakcję i nagrody
- Pomarańczowy – łączy dane oznaczone i nieoznaczone
- Turkusowy – tworzy własne sygnały nadzoru

generated with ChatGPT

Rekomendacje

- Drzewa decyzyjne i lasy losowe
- Modele z boostingiem (GBM, XGBoost)
- Przewidywanie szeregów czasowych
- Czym jest uczenie:
 - przez wzmocnienie (*reinforcement*)
 - częściowo nadzorowane (*semi-supervised*)
 - samonadzorowane (*self-supervised*)
- Twierdzenia o uniwersalności
- Idea *multi-layer perceptron* (MLP) + prosty model w PyTorch / TensorFlow
- Czym jest atencja + architektura transformatora (*transformer*)
- Architektura autoenkodera
- Nowoczesne API: Optuna, Ray, wandb, Pytorch Geometric, Pyro, TensorFlow Probability
- Używanie GenAI do generowania architektur i syntetycznych zbiorów danych